

Terraform 101

An overview of Terraform

Resources:

- <https://www.terraform.io/>
- <https://www.youtube.com/watch?v=iRaai1IBIB0>

What Is Infrastructure as Code?

Write and execute code to define, deploy, update, and destroy your infrastructure.

- Scripting (Bash, Python, Ruby, ...)
- Configuration management tools (Chef, Puppet, Ansible, SaltStack)
- Server Templating tools (Docker, Packer, ...)
- Orchestration tools (Kubernetes, Docker Swarm, AWS ECS, ...)
- Provisioning tools (Terraform, CloudFormation, Pulumi)

Some uses cases

- Automate Deployments
- Multi-cloud deployments
- Disposable environments
- Software Defined networking
- Immutable infrastructure
- Security automation
- Disaster Recovery and Backups

laC benefits

- Speed
- Documentation
- Version Control
- CI/CD easy integration
- Immutable architecture
- Testing
- Reuse
- Just coding

Why Terraform?

- Declarative
 - *Only define the desired state.*
- Immutable
- Idempotent
- Multi provider
- Multi platform (binary per architecture)
- Easy to install
- Community

How Terraform Works

- Open Source tool
- Written in Go programming language
- Single binary
- API Calls (to one or more providers)
- Configuration files. (code in Iac)

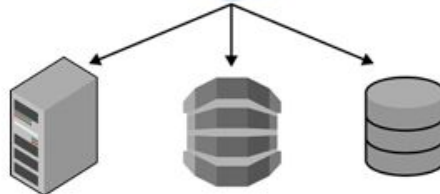
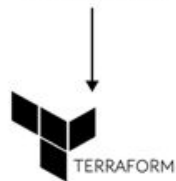
Installing Terraform

- Binaries
- brew
- tfswitch / or any script you prefer
- Docker

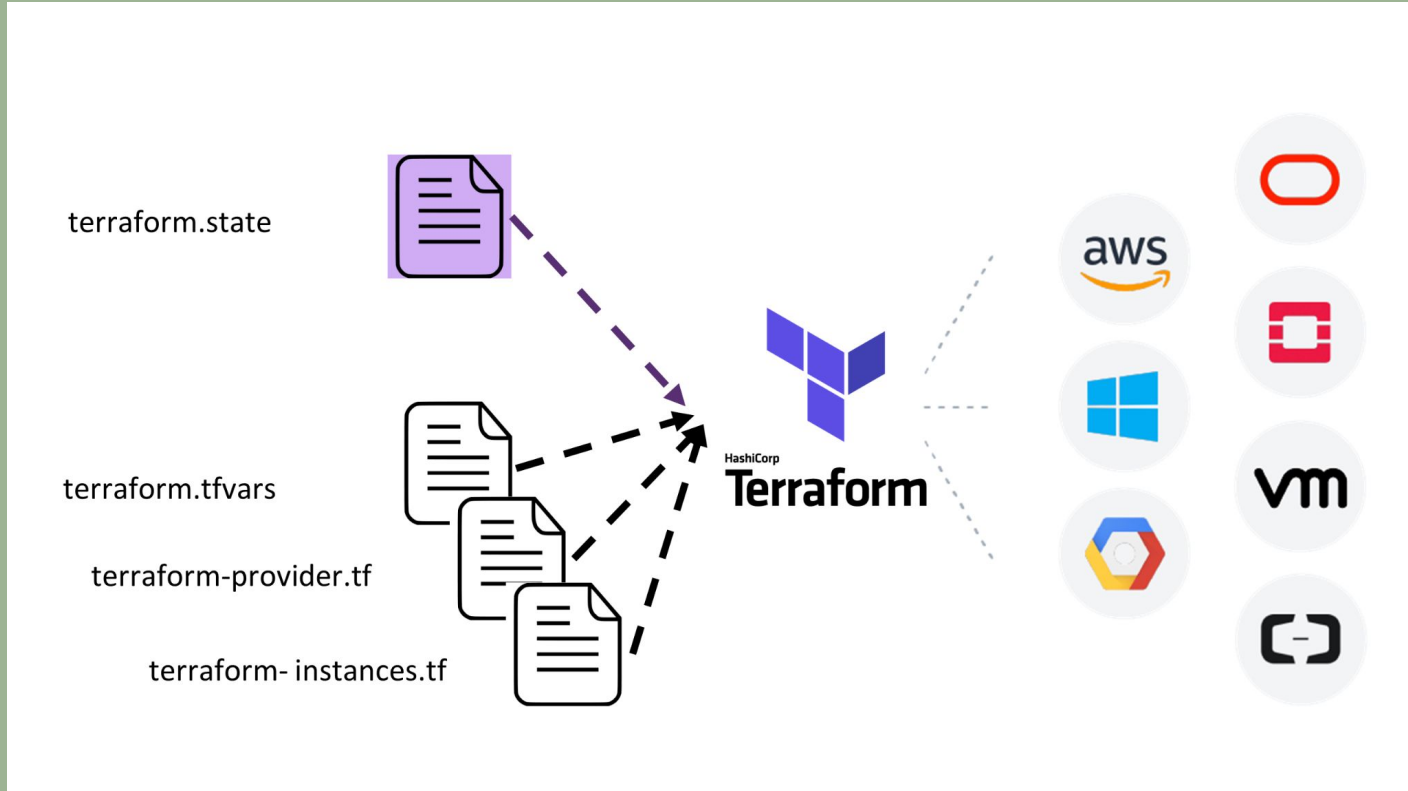
```
resource
"aws_instance" "a" {
  ami = "ami-40d28157"
}

resource
"aws_db_instance" "db"
{
  engine = "mysql"
  name = "mydb"
}
```

Terraform configuration



How Terraform Works



Terraform Project – Basic files

main.tf: where resources, providers, and configurations are defined.

variables.tf: Defines the variables.

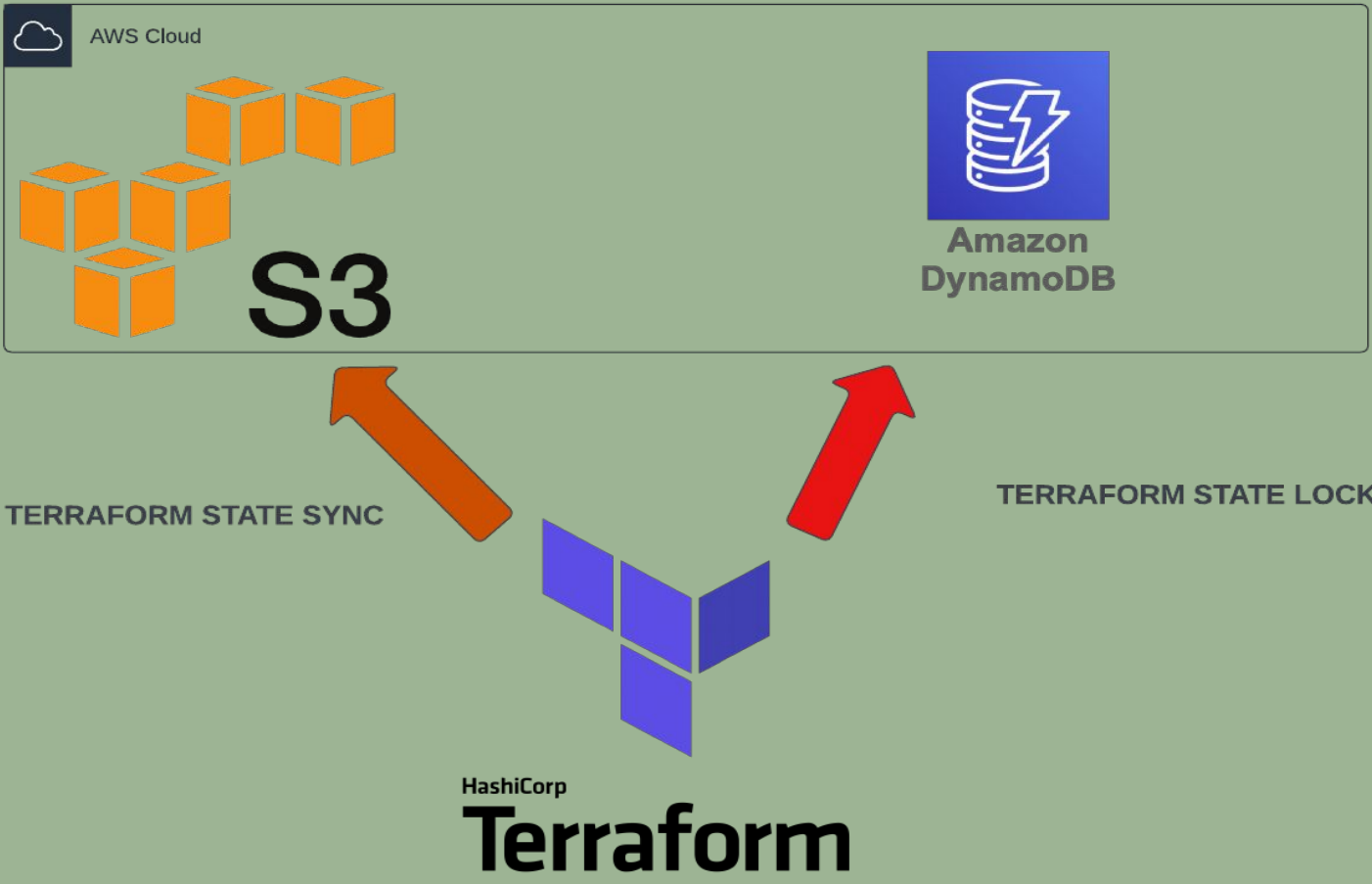
outputs.tf: Specifies the outputs that Terraform will return after the execution.

terraform.tfvars: Define specific values for the variables

provider.tf: Sometimes separated to define the provider (e.g., AWS, Azure) and its configuration settings.

Files and Directories

- File extensions
 - Plain text
 - .tf, .tf.json
- Text encoding.
- UTF-8
- Unix-style line ending (LF) over (CRLF).
- A terraform module consist of the top-level configuration.
 - Nested modules are treated as separate modules.
- A terraform module can use module calls to include other modules.
- Terraform runs in the context of a single root module.
 - A Terraform configuration consists of a root module and the tree of child modules.
 - In Terraform CLI, the root module is the working directory where Terraform is invoked.



How Terraform Works

```
resource "aws_instance" "example" {  
    ami          = "ami-0c55b159cbfaffe1f0"  
    instance_type = "t2.micro"  
}  
  
resource "google_dns_record_set" "a" {  
    name          = "demo.google-example.com"  
    managed_zone = "example-zone"  
    type          = "A"  
    ttl          = 300  
    rrdatas      =  
[aws_instance.example.public_ip]  
}
```

Resource Blocks

```
resource "aws_instance" "web" {  
  ami          = "ami-a1b2c3d4"  
  instance_type = "t2.micro"  
}
```

Terraform syntax HCL

```
resource "aws_vpc" "main" {  
  cidr_block = var.base_cidr_block  
}  
  
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {  
  # Block body  
  <IDENTIFIER> = <EXPRESSION> # Argument  
}
```

- The purpose of the Terraform language is declaring resources, which represent infrastructure objects.
 - Blocks are containers for other content.
 - Have block type, can have zero or more labels, and have a body that contains any number of arguments and nested blocks.
 - Arguments assign a value to a name.
 - Expressions represent a value, literally or by referencing and/or combining.

Data sources

Data sources allow Terraform use information defined outside of Terraform, defined by another separate Terraform configuration, or modified by functions.

```
# Find the latest available AMI that is tagged with Component = web
data "aws_ami" "web" {
  filter {
    name   = "state"
    values = ["available"]
  }

  filter {
    name   = "tag:Component"
    values = ["web"]
  }

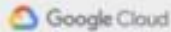
  most_recent = true
}
```


How Terraform Applies a Configuration

- **Create** resources that exist in the configuration but are not associated with a real infrastructure object in the **state**.
- **Destroy** resources that exist in the state but no longer exist in the configuration.
- **Update** in-place resources whose arguments have changed.
- **Destroy** and **re-create** resources whose arguments have changed but which cannot be updated in-place due to remote API limitations.

executables

Terraform Provider



executable



Terraform

INIT - PLAN - APPLY - DESTROY



filename.tf



Terraform Workflow

API



terraform.tfstate

Terraform block configuration

```
terraform {  
  required_providers {  
    aws = {  
      version = ">= 2.7.0"  
      source  = "hashicorp/aws"  
    }  
  }  
}
```

Backend Configuration

Each Terraform configuration can specify a backend, which defines exactly where and how operations are performed, where **state** snapshots are stored, etc.

```
terraform {  
  backend "remote" {  
    organization = "example_corp"  
  
    workspaces {  
      name = "my-app-prod"  
    }  
  }  
}
```

Providers

- Terraform relies on plugins called "providers" to interact with cloud providers.
- Terraform configurations must declare which providers they require.

Variables and outputs.

Input Variables serve as parameters for a Terraform module, so users can customize behavior without editing the source.

Output Values are like return values for a Terraform module.

Local Values are a convenience feature for assigning a short name to an expression.

Variable constraints restrict the type of value that will be accepted as the value for a variable. If no set then a value of any type is accepted.

- The supported type constraints keywords are:
 - string
 - number
 - bool
- The type constructors allow you to specify complex types:
 - list(<TYPE>)
 - set(<TYPE>)
 - map(<TYPE>)
 - object({<ATTR NAME> = <TYPE>, ... })
 - tuple([<TYPE>, ...])

Variable arguments

- default - A default value which then makes the variable optional.
- type - This argument specifies what value types are accepted for the variable.
- description - This specifies the input variable's documentation.
- validation - A block to define validation rules, usually in addition to type constraints.
- sensitive - Limits Terraform UI output when the variable is used in configuration. (true or false)

Variables

```
variable "image_id" {  
  type      = string  
  description = "The id of the machine image (AMI) to use for the server."  
}
```

```
resource "aws_instance" "example" {  
  instance_type = "t2.micro"  
  ami           = var.image_id  
}
```

Variables in command line

```
terraform apply -var="image_id=ami-abc123"  
terraform apply -var='image_id_list=["ami-abc123","ami-def456"]' -var="instance_type=t2.micro"  
terraform apply -var='image_id_map={"us-east-1":"ami-abc123","us-east-2":"ami-def456"}'
```

Variables in command line

```
terraform apply -var-file="testing.tfvars"
```

Variables in command line

```
export TF_VAR_image_id='ami-abc123'
```

```
export TF_VAR_availability_zone_names='["us-west-1b","us-west-1d"]'
```

Variable Definitions (.tfvars) Files

To set lots of variables, it is more convenient to specify their values in a variable definitions file (with a filename ending in either .tfvars or .tfvars.json) and then specify that file on the command line with `-var-file`:

```
terraform apply -var-file="testing.tfvars"
```

Variable from environment

```
$ export TF_VAR_image_id=ami-abc123  
$ terraform plan
```

Terraform loads variables in the following order, with later sources taking precedence over earlier ones:

- Environment variables
- The terraform.tfvars file, if present.
- The terraform.tfvars.json file, if present.
- Any *.auto.tfvars or *.auto.tfvars.json files, processed in lexical order of their filenames.
- Any -var and -var-file options on the command line, in the order they are provided.

Modules

```
module "servers" {  
    source = "./app-cluster"  
  
    servers = 5  
}
```



```
condition ? true_val : false_val
```



If `condition` is `true` then the result is `true_val`. If `condition` is `false` then the result is `false_val`.

A common use of conditional expressions is to define defaults to replace invalid values:

```
var.a != "" ? var.a : "default-a"
```



If `var.a` is an empty string then the result is `"default-a"`, but otherwise it is the actual value of `var.a`.

Count

- The **count** argument replicates the given resource or module a specific number of times with an incrementing counter. It works best when resources will be identical, or nearly so.

for_each

- The for_each meta-argument allows you to configure a set of similar resources by iterating over a data structure to configure a resource or module for each item in the data structure.

